

Full-Stack Developer

Dahee Kim

Engineer · Tinkerer · Explorer

Turning **curiosity** into **experiments**
Turning **experiments** into **products**

About Me.

The best part of building software is
a few lines of code can turn
frustration into convenience.

What excites me most is
seeing how software can solve everyday frustrations.

Whether I'm building with frontend, backend, or AI,
I enjoy creating products that make people's lives a little easier.

To me, great software starts with a real problem
and ends with a better experience.

// CORE KEYWORDS

01
System Thinking

02
Full-Stack Development

03
Operational Stability

04
AI & Continuous Learning

Core Strengths.

Four strengths defined not by the breadth of my stack, but by **how I build products**.

01 System Design

Instead of treating problems in isolation,
I design solutions from a whole-system perspective.

02 Full-Stack Development

From React, Next.js on the frontend to
Nest.js, Spring Boot, and FastAPI on the backend.
I can build the entire product from end to end.

03 Operational Stability

With Docker, AWS, and CI/CD automation experience,
I build reliable, scalable environments.

04 Exploring New Tech

Currently drawn to LLMs, agents, MCP, and automation,
exploring how emerging technologies can create real products and
better user experiences.

Experience.



Furence | Backend Engineer

2025.11 ~ 2026.02

Designed backend architecture for LG U+ call-recording system, implemented authorization logic, built a large-scale audio upload pipeline, and supported STT-based workflows.



ThinkStorm | Full-Stack Engineer

2024.04 ~ 2025.10

Designed architecture for Nest.js backend and Next.js frontend, with CI/CD automation and performance optimization experience.



Sherpasoft | Full-Stack Engineer

2023.07 ~ 2024.03

Developed enterprise DB performance management solutions and internal business automation systems.

// IN ONE LINE

An engineer with experience building enterprise-grade systems and shipping production-ready products.

Tech Stack.

From **UI** · **server** · **infrastructure** - the tools I use to connect every layer of a product.

// 01

Frontend

UI · Interaction · State

React.js

Next.js

CSS

HTML

// 02

Backend

API · Architecture · Logic

Nest.js

Spring Boot

Node.js

FastAPI

// 03

Data & Infra

Storage · Deploy · CI/CD

PostgreSQL

MySQL

MongoDB

Redis

Kafka

Docker

AWS

GitHub Actions



Natours

Eco-Friendly Tour Booking Platform · Full-Stack Web App



// OVERVIEW

A full-stack tour booking platform for discovering and reserving eco-friendly tours with ease.

Built with Node.js, Express, and MongoDB, with Pug-based SSR for fast loading and SEO optimization. Integrated Stripe for payments, Mapbox for interactive tour maps, AWS S3 for image storage, and SendGrid for transactional email - delivering a production-ready service from end to end.

// ARCHITECTURE & SECURITY

Modular MVC structure for maintainability and scalability, layered with production-ready security : Helmet, rate limiting, data sanitization (xss-clean, mongo-sanitize), and HPP protection.

// KEY FEATURES

- **Stripe Payments** — Secure card-payment flow for tour reservations
- **Mapbox Interactive Maps** — Visualized tour locations for intuitive discovery
- **JWT + Role-Based Access** — Differentiated user/admin permissions
- **AWS S3 Image Uploads** — Scalable media storage with CORS configuration
- **SendGrid Transactional Email** — Welcome emails and password reset workflows
- **Admin Dashboard** — Centralized control over tours, users, reviews, and bookings
- **Revenue Dashboard** — AmCharts-powered booking history and revenue analytics



WeddingYou

Wedding Planner Matching Platform · Full-Stack Web App

 GitHub

 Live Demo 

// OVERVIEW

A planner-matching platform that connects every step of the wedding journey.

Built with Spring Boot, React, and PostgreSQL - designing and implementing every core feature required for a wedding platform from user-planner matching to payments, image uploads, and email notifications. Migrated from MySQL (Railway) to PostgreSQL (Fly.io) along the way, gaining hands-on experience with database migration.

// ARCHITECTURE

Domain model expressed through Spring Data JPA OneToMany · ManyToOne · ManyToMany relationships. Full-stack orchestrated via Docker Compose, deployed with Netlify (frontend) and Fly.io (backend).

// KEY FEATURES

- **Pre-Quote Based Matching** — Clients submit ideal wedding plan; planners review and match
- **Mutual Like/Swipe Matching** — Intuitive two-way matching UX
- **PortOne (Iamport) Payments** — Real-time payment processing & confirmation
- **SMTP Email Notifications** — Password reset workflows
- **AWS S3 Image Uploads** — Storage for profiles, wedding photos, vendor logos
- **Optimistic UI** — Instant feedback on RSVP and guest list edits
- **Role-Based Authentication** — User / Admin / Planner permission separation



Netflix and Chill

Netflix-Inspired Streaming Platform · Full-Stack Web App

 [GitHub](#)

 [Live Demo](#) ↗

// OVERVIEW

A Netflix-inspired full-stack streaming platform - supporting both user and admin experiences.

Built with React + Redux on the frontend, Spring Boot on the backend, and PostgreSQL (Supabase) for storage. Users discover, search, and favorite movies, while admins manage content, users, and comments through a comprehensive dashboard.

// DEPLOY & SECURITY

Dockerized backend (Render) and frontend (Netlify) for scalable cloud deployment. JWT-based authentication with User/Admin role separation for secure access control.

// KEY FEATURES

- **Movie Discovery & Search** — Intuitive UI with details and trailers
- **Personal Favorites List** — Local persistence for quick rewatching
- **JWT + Role-Based Authentication** — Separated User/Admin token flow
- **Admin Dashboard** — Unified control over users, content, comments, and movies
- **Dockerized Full-Stack** — Render · Netlify · Railway automated cloud deployment
- **Responsive UI** — Dynamic routing and real-time updates on desktop and mobile
- **Redux + Context API** — Separated global and scoped state management



Wetube

YouTube-Style Video Sharing Platform · Full-Stack Web App



// OVERVIEW

A full-stack video streaming platform that recreates YouTube's core experience.

Built with Node.js, Express, MongoDB, and Pug SSR - implementing video browsing, playback, uploads, and commenting from scratch. Multer + AWS S3 powers a secure media-processing pipeline, and Webpack handles production-grade bundling.

// AUTH

Designed JWT-based token issuance, protected routes, and refresh/logout flows from scratch along with GitHub OAuth social login for a flexible authentication experience.

// KEY FEATURES

- **Record · Upload · Playback** — In-browser recording + streaming playback
- **Multer + AWS S3** — Scalable file storage with real-time upload previews
- **Live Commenting System** — Two-way interaction below each video
- **JWT + GitHub OAuth** — Token-based auth integrated with social login
- **Pug SSR** — Server-side rendering for faster initial loads
- **Webpack Bundling** — Optimized JS · CSS · asset management for production
- **Fly.io Deployment** — Stable global edge hosting



Forkify

Vanilla JavaScript Recipe Search SPA · Frontend Project

 [GitHub](#)

[● Live Demo ↗](#)

// OVERVIEW

An interactive recipe search app built entirely with vanilla JavaScript - no frameworks.

Powered by the Forkify API, users can search thousands of recipes and view detailed ingredients, cooking time, and servings. A modular, MVC-based architecture demonstrating that maintainable code can be achieved without frameworks.

// WHY THIS PROJECT

A learning project to master JavaScript fundamentals - event delegation, module patterns, async/await, DOM abstraction - and **become a developer who truly understands how their own code works.**

// KEY FEATURES

- **Recipe Search** — Keyword search via Forkify API
- **Detailed Views** — Ingredients, cook time, servings, and publisher info
- **Dynamic Serving Adjustment** — Real-time ingredient recalculation
- **Bookmark Feature** — Persistent bookmarks using localStorage
- **Mobile-First Design** — Fully responsive across all devices
- **MVC Architecture** — Cleanly separated Model · View · Controller layers
- **Netlify Deployment** — Auto-deploy on every push to main



ThinkStorm

Where Innovation Meets Collaboration · Developer Collaboration Platform



// OVERVIEW

A collaboration platform where developers, designers, and entrepreneurs turn ideas into reality together.

ThinkStorm is more than a project management tool - it's a new kind of career platform where **"your profile becomes your resume."**

Designed the flow where projects → collaborator matching → completed work automatically accumulates into a living portfolio.

// MY ROLE

Designed the **architecture** for Nest.js backend and Next.js frontend, with CI/CD automation and performance optimization. Personally designed the data models and authorization flows of a collaboration-focused platform.

// KEY FEATURES

- **Project Discovery & Collaboration** — Category/tag search, team invites, channels
- **Dynamic Profile** — A living portfolio that grows with every completed project
- **Weekly Challenges** — Themed competitions + leaderboards + rewards to drive growth
- **Learning & Development** — Project-based bootcamps + certification (upcoming)
- **Resume-less Hiring** — Your profile and completed projects ARE your application
- **Real-time Notification System** — Project updates · messages · new opportunities
- **Support System** — Operational structure for user help requests



Interview App

AI Interview Practice Application · 5-Day Sprint Project



// OVERVIEW

An AI interface experiment - chat with an AI interviewer in real time and receive instant feedback on your answers.

Built on FastAPI (backend) + Streamlit (frontend), connecting OpenAI Agents SDK flows (triage / analyze / revise / final) through SSE (Server-Sent Events) streaming. The focus was on transforming raw CLI-based AI interactions into a polished product experience.

// 5-DAY SPRINT

Day 1 Streamlit chat UI · **Day 2** FastAPI SSE API · **Day 3** Agents SDK integration · **Day 4** Multi-page navigation · **Day 5** Error handling · feedback · UX polish

// KEY FEATURES

- **Real-time SSE Streaming** — FastAPI `StreamingResponse` + Streamlit placeholder
- **Agent Orchestration** — Agents SDK driving triage / analyze / revise / final flow
- **Function Calling** — Tool invocation via `/resume/questions` for context extension
- **Feedback UX** — `st.feedback("thumbs")` + backend `/feedback` collection
- **Multi-page Navigation** — Separated interview / resume / settings areas
- **Health Check + Error Handling** — Backend health checks with friendly error UX
- **Conversation Search + Markdown Rendering** — Filtering + readable code blocks



Picture Diary

LLM-Powered Image/Video Diary Pipeline · AI Pipeline Project



// OVERVIEW

A multimodal pipeline that transforms diary entries into scenes and visualizes them with LLM-driven image and video generation models.

GPT automatically divides each diary entry into four scenes - gpt-image-2, FLUX-schnell, and Kling each produce images and videos. Beyond content generation, this project engineers **cost tracking, domain prompts, and A/B testing**.

// COST & LATENCY

Over 5 days, generated 15 images/videos for a total of **\$0.139**, with P95 latency measured at **2.4s** - measuring and optimizing both cost and latency.

// KEY FEATURES

- **Scene Decomposition Pipeline** — GPT splits diary into 4 scenes automatically
- **Multi-Model Routing** — gpt-image-2 · FLUX-schnell · Kling selected based on the use case
- **Domain Prompt System** — travel · product · emoji style dictionaries per domain
- **A/B Testing + P95 Measurement** — Quantitative latency & quality comparison by seed
- **Automated Cost Reporting** — `cost_report.md` · `ab_test_results.json` generation
- **Modular Agent Structure** — `scene.py` · `image.py` · `video.py`
- **Date-based Result Archive** — `outputs/YYYY-MM-DD/` auto-organized



Watchers



CCTV-Based Industrial Safety Auto-Labeling & Detection · AI · MLOps Team Project

// OVERVIEW

An industrial-safety MLOps pipeline that auto-detects and labels workplace hazards in CCTV footage.

As the **team lead** of a 4-person team, I owned end-to-end system integration and architecture → backend, MCP server, GPU server, and frontend. Designed an auto-labeling pipeline fusing Grounding DINO zero-shot detection with YOLO Active Learning, orchestrating 4 services through **35 FastAPI endpoints · 9 database tables**.

// SAFETY-FIRST MODEL GATING

Designed a Gold Set gating system: no retraining without human review, and models only promote when the safety-critical class (`no_helmet`) recall improves.

RECALL **66.4%** → **78.3%** ↑ +11.9pp
Improved via SGD · AdamW · MuSGD comparative experiments

// KEY FEATURES

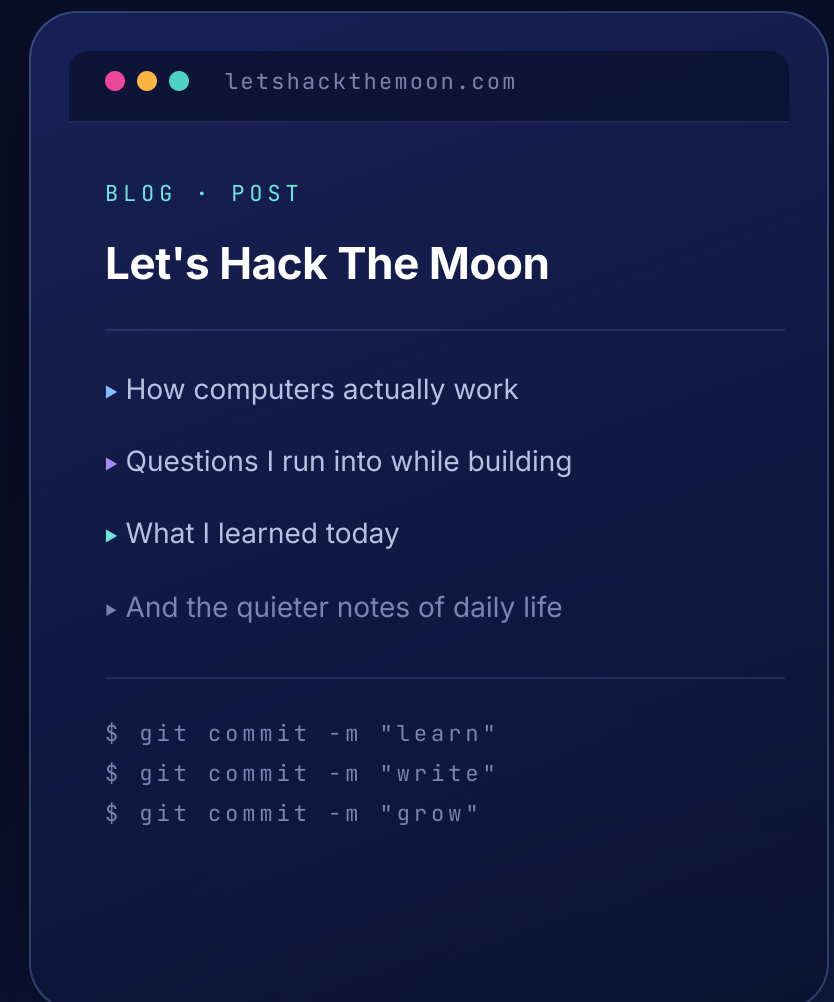
- **Auto-Labeling Pipeline** — Motion diff → DINO + Active Learning dual draft → CVAT review → dataset build
- **MCP Natural-Language Control** — OpenAI native MCP integration; trigger training, labeling & KPIs via 11 tools
- **Gold Set Promotion Gate** — Auto-promote only when safety metrics don't regress; audit-logged
- **Class Index Integrity Lock** — Prevents label-index drift from baking into model weights
- **4 Independent Service Architecture** — Backend · GPU-backend · MCP-backend · Electron (REST + callback)
- **Team Leadership & System Integration** — Led 4-person team. Owned end-to-end architecture

Writing & Branding.

“ Learning doesn't stop at **implementation**.
organizing and explaining what I learn
is also part of being a developer.

On my blog, I write about how computers actually work, the questions I run into while building, and what I've learned - all in my own voice.

I hope my experiences can make someone else's learning journey a little easier.



Let's Build Something Meaningful.

I believe great products are built through curiosity, continuous learning, and collaboration.

I enjoy asking questions, learning from others, and turning everyday frustrations into products that make life a little easier.

That's the kind of developer I strive to be.

01 · PORTFOLIO

Personal Site

daheekim.app

02 · BLOG

Writing Journal

letshackthemoon.com

03 · GITHUB

Code Repository

github.com/kimdahee0815

04 · LINKEDIN

Professional

linkedin.com/in/dahee-kim-6aaa5b267